

Multilingual GenAI Voice Assistant for Customer Care

Complete Solution Approach — Agentic Architecture

Fine-Tuning · Orchestrator & Sub-Agents · Tools & RAG · Guardrails · Live-Model Alternative

The vision: a customer speaks in ANY language/accent and the assistant understands, acts, and answers in that same language — for telecom self-service (bill queries, plan changes, complaints, coverage). Underneath sits a shared **orchestrator agent** that routes each request to specialized **sub-agents**, each with its own tools and **RAG knowledge base**, wrapped in **guardrails** for safety and correct hand-off.

Two solution tracks: (A) a **custom STT** → **Agent(LLM+RAG)** → **TTS pipeline** you fine-tune and control, and (B) a **live native-audio model** (e.g., Gemini Live API) that collapses STT+TTS into one low-latency model.

Dataset in play: Mozilla Common Voice — a multilingual speech corpus (250+ languages, accents, CC0) providing paired **audio + transcript + speaker demographics** for fine-tuning the speech models.

1. Solution Overview — The End-to-End Voice Loop

The system is a layered voice agent. Audio comes in, is understood, grounded on company knowledge, acted upon via tools, and spoken back — with guardrails deciding when to hand off to a human.

High-Level Architecture (Layers)

Layer	Responsibility
1. Voice I/O	Captures customer audio and streams the spoken reply back (telephony / app mic).
2. Speech-to-Text (STT)	Fine-tuned ASR transcribes multilingual, accented speech → text + language tag.
3. Orchestrator Agent	Detects intent, manages the dialogue, routes to the correct sub-agent.
4. Sub-Agents	Specialists (Billing, Plans, Complaints, Coverage/Technical) with their own tools + RAG.
5. Tools & RAG	Function-calling tools (backend APIs) + retrieval over knowledge bases.
6. Guardrail / Policy	Safety, confidence checks, sensitive-intent detection, human hand-off.
7. Text-to-Speech (TTS)	Converts the grounded reply into natural voice in the customer's language.
8. Data Stores	Vector DBs (RAG), operational DBs/APIs (billing, CRM), session/transcript logs.

2. Model Fine-Tuning — What & How

The pipeline has three model layers. Spend fine-tuning effort where it moves accuracy most for multilingual, accented telecom speech.

Layer	Fine-tune?	Why / What to adapt
STT / ASR	YES (high value)	Accents, code-mixing ('Hinglish'), telecom jargon (data pack, top-up, IMEI) are where off-the-shelf ASR fails most — top priority.
LLM (agent brain)	USUALLY NOT	Prefer RAG + prompt engineering. Knowledge comes from RAG, not weights. Light instruction-tuning only if tone/format must be tight.
TTS	OPTIONAL	Only for a brand voice, a missing language/accent, or correct domain-term pronunciation. A strong multilingual base TTS is often enough.

2A. Preparing the Common Voice Data

1. **Filter languages/accents** for your customer base (e.g., Hindi, Tamil, English-IN).
2. **Clean transcripts** — normalize casing/numbers/punctuation; drop low-quality clips.
3. **Resample to 16 kHz mono** (standard Whisper input).
4. **Split train/val/test** with disjoint speakers for true generalization.

2B. Fine-Tuning STT (Whisper + LoRA)

Technique: PEFT with LoRA + int8/bitsandbytes quantization. Freeze Whisper weights; inject trainable low-rank matrices into attention (W_q , W_k , W_v , W_o) and feed-forward layers. Only ~1–5% of params train — fits <8 GB VRAM, ~60 MB adapters, no added latency, avoids catastrophic forgetting.

- **Advanced multilingual:** one LoRA “language expert” per language, fused or distilled → ~10–15% relative WER gains by reducing cross-language interference.
- **Setup:** Seq2Seq cross-entropy, AdamW, LR $\approx 1e-4$, rank $r \approx 16-32$. Metric: Word Error Rate (WER).

Whisper-large-v2 (int8) → freeze → add LoRA ($r=32$) → train on Common Voice (16kHz) → eval WER → save ~60MB adapter

2C. Fine-Tuning TTS (only if needed)

- **Zero-shot cloning (no training):** give a ~6-sec reference clip (XTTS-style) — captures tone, may miss pacing.
- **Fine-tuning (best quality):** 5+ min of clean single-speaker audio; toolchain auto-transcribes (Whisper) and trains the conditioning encoder. LoRA-16bit or full FT — TTS models are small.
- **Missing language:** fine-tune on that Common Voice locale for correct phonetics/prosody. Metric: MOS + pronunciation checks.

3. Agentic App Structure — Orchestrator, Sub-Agents, Tools & RAG

The system uses a **supervisor (orchestrator) + specialist sub-agents** pattern. The orchestrator classifies intent and delegates to a focused sub-agent. Each sub-agent owns specific **tools** (backend API calls) and consumes a **RAG retriever as a tool** to ground answers in the operator's knowledge.

3.1 The Orchestrator Agent

- **Role:** takes the STT transcript + language, detects intent (billing / plans / complaint / coverage), extracts entities (account no., plan name), and routes to the right sub-agent. Maintains dialogue state and language context.
- **RAG:** none directly — it delegates retrieval to sub-agents.

3.2 Sub-Agents, Their Tools & RAG

A) Billing & Payments Agent

- **Purpose:** bill balance, due dates, last payments, charge explanations, payment links.
- **Tools:** getBalance(), getBillBreakup(), getDueDate(), sendPaymentLink(), explainCharge().
- **RAG (as a tool):** YES → **Billing-Policy RAG** (tariff rules, late-fee/roaming policies, charge glossary) to explain charges accurately.

B) Plans & Offers Agent

- **Purpose:** recommend/compare plans, activate/change a plan, explain add-ons and offers.
- **Tools:** listPlans(), comparePlans(), changePlan(), activateAddOn(), checkEligibility().
- **RAG (as a tool):** YES → **Product-Catalog RAG** (current plans, prices, offer T&Cs) so recommendations are current and compliant.

C) Complaints & Service-Request Agent

- **Purpose:** log complaints, check ticket status, troubleshoot common issues, set expectations on resolution.
- **Tools:** createComplaint(), getTicketStatus(), runTroubleshootFlow(), escalateToHuman().
- **RAG (as a tool):** YES → **Support-KB / FAQ RAG** (troubleshooting guides, known-issue articles, SLA policy).

D) Coverage & Technical Agent

- **Purpose:** network coverage, outage status, device/APN settings, SIM/eSIM help.
- **Tools:** checkCoverage(pincodes), getOutageStatus(), getDeviceSettings(), guideSimSwap().
- **RAG (as a tool):** YES → **Technical-KB RAG** (device/APN guides, coverage FAQs) + live outage API.

3.3 Shared Speech Services (not sub-agents, but pipeline components)

- **STT Service:** fine-tuned Whisper — audio → text (feeds the orchestrator).
- **TTS Service:** multilingual TTS — final text reply → voice.

4. RAG Knowledge Bases — Contents & Which Sub-Agent Uses Them

Each RAG is a vector index built from operator documents. The table maps **knowledge base** → **contents** → **consuming sub-agent**.

Knowledge Base (RAG)	What's in it	Used by (sub-agent)
Billing-Policy RAG	Tariff rules, late-fee/roaming policy, charge glossary, refund rules	Billing & Payments
Product-Catalog RAG	Current plans, prices, add-ons, offer terms & eligibility	Plans & Offers
Support-KB / FAQ RAG	Troubleshooting guides, known-issue articles, SLA / complaint policy	Complaints & Service-Request
Technical-KB RAG	Device/APN setup, SIM/eSIM guides, coverage FAQs	Coverage & Technical
Compliance/Policy RAG	Regulatory scripts, consent language, do/don't-say rules	Guardrail layer (all agents)

How RAG Works (per sub-agent)

- **Offline (indexing):** chunk the operator's docs → embed with a multilingual embedding model → store vectors in a vector DB (one namespace per knowledge base).
- **Online (query time):** the sub-agent embeds the user's query → semantic search over its OWN knowledge base → retrieves top-k passages → the LLM composes a grounded, policy-compliant reply in the customer's language.

Why scoped RAG per sub-agent: keeps retrieval precise (a billing question searches billing policy, not device guides), reduces hallucination, and makes each agent independently testable.

5. Guardrails & Human Hand-off

Guardrails run at multiple layers so the assistant stays safe, compliant, and knows when to escalate.

Layer 1 — Input Guardrails

- Language/confidence check on the STT output; if the transcript confidence is low or audio is unclear, ask to repeat or hand off.
- Detect abusive/fraudulent input and identity-verification needs before acting on an account.

Layer 2 — Action Guardrails (tool authorization)

- **Sensitive tools require verification:** `changePlan()`, `sendPaymentLink()` run only after identity is confirmed.
- **Compliance/Policy RAG** enforces mandated scripts/consent language before certain actions.

Layer 3 — Output Guardrails

- Scan the generated reply for policy violations, wrong pricing, or PII leakage before it is spoken.
- Ground-truth check: the answer must be supported by retrieved RAG context, else refuse/hand off.

Layer 4 — Human Hand-off

- Trigger on: low confidence, sensitive intent (billing dispute, cancellation), detected frustration (esp. via affective cues), or explicit request for an agent.
- Transfer WITH full context — transcript, intent, entities, actions attempted — so the human doesn't start cold.

6. End-to-End Request Flows

6.1 Customer asks (in Hindi): “Mera bill itna zyada kyun hai?” (Why is my bill so high?)

1. **STT** transcribes the Hindi audio → text + language=hi.
2. **Orchestrator** detects intent=billing, entity=current bill → routes to Billing & Payments Agent.
3. Billing Agent calls **getBillBreakup()** + queries **Billing-Policy RAG** to explain each charge.
4. **Guardrail** verifies identity before revealing account details; output checked for correct figures.
5. **TTS** speaks the grounded explanation back in Hindi.

6.2 Customer asks: “Switch me to an unlimited plan”

1. **Orchestrator** intent=plan-change → Plans & Offers Agent.
2. Agent calls **comparePlans()** + **Product-Catalog RAG** to find eligible unlimited plans, then **checkEligibility()**.
3. **Guardrail** changePlan() requires identity verification + mandated consent script (Compliance RAG).
4. On confirmation, **changePlan()** executes; TTS confirms in the customer's language.

6.3 Sensitive / low-confidence case → hand-off

1. Customer is angry and demands cancellation (a sensitive intent).
2. **Guardrail** detects frustration + sensitive intent → does NOT auto-cancel.
3. Assistant hands off to a human agent WITH transcript, intent, and account context.

7. Alternate Approach — Live Speech-to-Speech Model (No STT/TTS)

Idea: replace the STT + TTS models with a single **native-audio “live” model** — e.g., **Gemini Live API (Gemini 2.5 Flash Native Audio)** — that hears raw audio and speaks back through one low-latency model. The orchestrator/sub-agent/RAG layer stays; only the speech front-end changes.

Why It's Compelling

- **Removes the multi-stage pipeline:** processes raw audio in a single pass, cutting the STT→LLM→TTS turn-taking latency.
- **Seamless multilingual:** switches languages with no pre-configuration (HD voices across ~24 languages).
- **Affective dialog:** interprets tone/emotion and can de-escalate a stressed call empathetically.
- **Natural barge-in:** customer can interrupt mid-sentence; proactive-audio/VAD handles it.
- **Built-in tool use:** robust function calling + grounding, so it still drives your sub-agent tools & RAG.

How the Agentic System Plugs In

1. **Connect:** open a stateful WebSocket (WSS) session; audio in @16 kHz PCM, audio out @24 kHz.
2. **Converse:** customer speaks; the model understands and responds in voice natively (transcripts still available for logging).
3. **Ground with RAG + tools:** system instructions + function calling let it call your same backend tools (getBalance, changePlan) and RAG retrievers.
4. **Guardrail + hand-off:** same policy layer applies; transfer to human on sensitive/low-confidence cases.

```
async with client.aio.live.connect(model="gemini-live-2.5-flash-native-audio",
config={"response_modalities":["AUDIO"], "tools":[getBalance, changePlan, ragSearch]}) as
session:
    # stream mic audio in -> receive spoken response out
```

Pipeline vs. Live Model — Trade-offs

Aspect	Custom STT+Agent+TTS Pipeline	Live Native-Audio Model
Latency	Higher (3 sequential stages)	Very low (single pass)
Control / customization	Full — you fine-tune each model	Less — rely on provider's model
Multilingual / accents	Needs STT fine-tuning per language	Built-in, auto-switching
Emotion / barge-in	Extra engineering (VAD, prosody)	Native (affective, barge-in)
Data privacy / on-prem	Can self-host	Managed cloud service

Aspect	Custom STT+Agent+TTS Pipeline	Live Native-Audio Model
Best when	Custom voice, on-prem, niche languages, full control	Fastest natural experience, quick to build

Recommendation

Prototype fast with the Live model to prove the natural, low-latency experience for the demo. In parallel, keep the **custom fine-tuned pipeline** for full control, on-prem deployment, brand-voice ownership, and niche Indian language/accent coverage. A **hybrid** — Live model as the conversational front-end, your fine-tuned Whisper as fallback/logging transcriber, and the SAME sub-agent + RAG backend for both — gives the best of both worlds.

8. Summary — Why This Design Works

- **Clean separation of concerns:** **STT** hears, the **orchestrator + sub-agents** think and act, **TTS** speaks.
- **Scoped sub-agents + scoped RAG:** each specialist (Billing, Plans, Complaints, Coverage) grounds on its own knowledge base — precise, testable, low-hallucination.
- **Guardrails at every layer:** identity checks, compliance scripts, output validation, and human hand-off keep it safe and policy-compliant.
- **Future-proof:** the same agent/RAG backend works whether you run the custom pipeline or a live native-audio model — so you can start simple and swap the speech layer later.